VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING
MICRO PROJECT REPORT

Project Title : Distributed Cache with Consistent Hashing (FlashMart: a flash-sale shop on a replicated gRPC cache and a SQLite database)

SDG Aligned & Logo : SDG 4 – Quality Education (indirect educational contribution); SDG 9 – Industry, Innovation and Infrastructure (conceptual relevance)





Project / Course Details	To be filled by students
Course / Subject Name	Distributed Computing Lab (Micro Project)
Course Code / Semester / Academic Year	26CMPE742 / Semester VII / Academic Year 2026–2027
Faculty Incharge / Project Mentor	[Faculty mentor name]
Project Group No. (if applicable)	[Group number]
Submission Date	[dd Month 2026]

Project Team Details

Student Name	Roll No.	Class / Division	Institute Email ID
[Student name 1]	[Roll]	[Class/Div]	[institute email]
[Student name 2]	[Roll]	[Class/Div]	[institute email]
[Student name 3]	[Roll]	[Class/Div]	[institute email]

Department of Computer Engineering

Academic Year: 2026–2027

Abstract

Online services that read the same data again and again from one database slow down under load. A cache solves this, but a single cache server is both a single point of failure and a capacity limit. This project implements a **Distributed Cache with Consistent Hashing** and demonstrates it with FlashMart, a simulated flash-sale shop backed by a real SQLite database. Cache servers run as separate operating-system processes and are called by a coordinator over gRPC. Keys are placed on a hash ring with 40 virtual nodes per server and stored on two servers; failures are found by heartbeats, reads fail over to the replica, and the cluster rebalances when a server joins or leaves. In the tested setup (200 products, 24 concurrent simulated users, a database made to take about 180 ms per query), a cache hit took 0.88 ms against 188.2 ms for a database read. The cache answered 75.6% of 1000 flash-sale requests and cut the mean response time from 272.8 ms to 41.6 ms. Adding a server moved 32 of 200 keys (16%) with consistent hashing, whereas plain hash-modulo-N placement would have moved 151 (75.5%). Killing a server during continuous reads produced no failed requests in two test runs. A web interface explains and visualises each step, and backend logs, database tables and a read-only SQL console record every cache hit and miss. The implementation is an educational prototype on one machine, not a production cluster.

1. Introduction

1.1 Background and Motivation

Distributed computing spreads work and data over several computers or processes that cooperate by exchanging messages. A common use is to place a cache in front of a database. A cache keeps recently used answers in fast memory, so repeated requests avoid the slower database. When one cache server is not enough, or when its failure must not take the service down, the cache itself is distributed over several servers. That raises three questions: which server holds which key, what happens when a server fails, and what happens when servers are added or removed. This project answers them with consistent hashing for placement, replication for fault tolerance, heartbeats for failure detection, and rebalancing for membership changes, and it applies the Distributed Computing concepts of remote procedure calls, partitioning, replication, failure handling and scalability.

1.2 Problem Context

Consider an online shop during a flash sale. Thousands of shoppers open the same few product pages at once, and each page needs a database read. If every read goes to the database, response time grows with load. If one cache server is used, its failure sends all reads back to the database at once. The obvious way to share keys among N cache servers is `hash(key) % N`. It works until N changes: going from three servers to four changes the answer for about three quarters of the keys, so most of the cache becomes useless at the same moment (a cache stampede). Consistent hashing places servers and keys on a circle and changes the owner of only about 1/N of the keys when a server joins or leaves. Showing this difference on a running system is the central aim of the project.

1.3 Intended Users / Stakeholders

Primary stakeholders are students of Distributed Computing, laboratory instructors who evaluate the concepts, and developers who want to see how distributed caches place and replicate data. The system is meant for teaching and demonstration. It is not intended to serve production traffic.

2. Problem Definition, Objectives and Scope

2.1 Problem Statement

Design, implement and evaluate a distributed cache whose servers are independent processes communicating by RPC, which (a) distributes keys by consistent hashing, (b) keeps answering reads when a server fails, (c) works in front of a real database, and (d) lets an observer see and measure how placement, replication, failover and rebalancing behave. The system must run on a student laptop and be demonstrable live.

2.2 Objectives

1. Build cache servers as separate processes that expose Get, Put, Delete, Keys, Ping, Stats and Clear over gRPC, with LRU eviction and time-to-live (TTL).
2. Implement a consistent hash ring with virtual nodes and a clockwise lookup that returns an owner and a replica for every key.
3. Replicate every key on two servers and keep serving reads when a server is killed.
4. Detect failures automatically with heartbeats and rebalance after servers join, leave or fail.
5. Place the cache in front of a real SQLite database with read-through caching and invalidation on writes.
6. Measure latency, hit ratio, database load, keys moved (against hash-modulo-N), balance across servers and failover behaviour.
7. Provide a web interface and backend logs that make every step visible for a live demonstration.

2.3 Scope and Deliverables

Included: three to eight cache server processes on one machine; consistent hashing with adjustable virtual nodes; replication factor 2; heartbeat failure detection; automatic rebalancing; SQLite database with `products`, `access_log` and `query_log` tables; flash-sale traffic generator; six-tab web interface; read-only SQL console; `show_db.py` inspector; automated tests for the hash ring. **Excluded:** a replicated coordinator, quorum or versioned writes, persistent cache storage, authentication and encryption, real multi-machine deployment testing, autoscaling and production monitoring. **Deliverables:** source code, running demonstration, this report, README and EXPLAINED.md documents.

2.4 Assumptions and Constraints

All cache servers run on one laptop, each as its own process with its own memory and TCP port, so network delay, packet loss and partitions are not exercised. The database is one local SQLite file; each query is delayed by about 180 ms plus 4 ms for every query already in flight, to imitate a database on another machine that slows down under load. The product catalogue is synthetic (200 rows, fixed random seed 7). Traffic is simulated by 24 concurrent users with skewed popularity, so a few products receive most requests. No personal or private data is used. Results vary slightly between runs because of thread timing.

3. Background / Literature Review

The project applies established ideas rather than proposing a new algorithm. Consistent hashing was introduced to spread web caches without remapping most keys when servers change [1]. Amazon Dynamo combined consistent hashing with virtual nodes, replication on successor nodes and decentralised failure detection [2]. Large web services place memcached-style caches in front of databases and delete cached entries when the database changes [3]. gRPC provides the remote procedure call layer with Protocol Buffers contracts [4], and SQLite supplies an embedded SQL database [5]. Flask serves the web interface and API [6]. This project does not claim a formal survey of research literature; the sources below document the concepts and tools it uses.

Source / Year	Approach or Key Finding	Strength / Limitation	Relevance to This Project
Karger et al., "Consistent Hashing and Random Trees," 1997 [1]	Maps servers and keys to a circle so only a fraction of keys move when servers join or leave.	Strong theoretical basis; one point per server can give uneven load.	Basis of the ring; virtual nodes are used to even the load.
DeCandia et al., "Dynamo," 2007 [2]	Key-value store using consistent hashing with virtual nodes, replication to successor nodes and failure detection.	Highly available; much richer than needed here (versioning, quorums, hinted handoff).	Source of the owner-plus-successor replica idea, implemented in a simplified form.
Nishtala et al., "Scaling Memcache at Facebook," 2013 [3]	Describes a look-aside cache in front of databases and invalidation of cached data on writes.	Proven at very large scale; many of its concerns lie beyond a laboratory prototype.	Supports the read-through read path and delete-on-write invalidation used here.
gRPC documentation [4]	Defines services in .proto files; generates client and server code; uses HTTP/2.	Efficient and language-neutral; requires code generation and a running server.	Used for all calls between the coordinator and the cache servers.
SQLite documentation [5]	Serverless SQL database in one file, with write-ahead logging.	Simple and reliable; a single local file, not a distributed database.	Used as the real source of truth behind the cache and for the request logs.

3.1 Identified Gap and Project Contribution

Most explanations of consistent hashing stop at a diagram. The contribution here is an integrated, runnable teaching system in which the same cluster can be explored in three ways: a step-by-step lab that shows the MD5 hash, the position on the ring, the clockwise walk, the owner, the replica, the cache lookup and the SQL query for any key; a before-and-after preview that puts consistent hashing next to `hash % N` on the same 200 keys; and a real database with a request log, so that every cache hit and miss can be inspected from the backend. The report separates what is real (separate processes, gRPC calls, SQL queries, measured timings) from what is simulated (the database delay and the shoppers).

4. Proposed Solution and System Design

4.1 Proposed Solution

FlashMart is a shop whose product pages are read through a distributed cache. A coordinator program receives each request, hashes the product key to a position on the ring, finds the owner server and its

replica, and asks them over gRPC. A hit is returned immediately. On a miss the coordinator runs a `SELECT` on the SQLite database, returns the row and stores copies on the owner and the replica. A heartbeat thread watches every server; when a server stops answering, reads go to the replica at once and the coordinator removes the server from the ring and rebuilds the missing copies. A browser interface lets the user explore the ring, follow one key, shop, inspect the database and control the cluster.

4.2 System Architecture / Block Diagram / Conceptual Design

The system has four parts: the browser interface, the coordinator, the cache servers and the database. The coordinator is the only component that knows the ring. Each cache server is an independent process that stores entries in memory and knows nothing about the others.

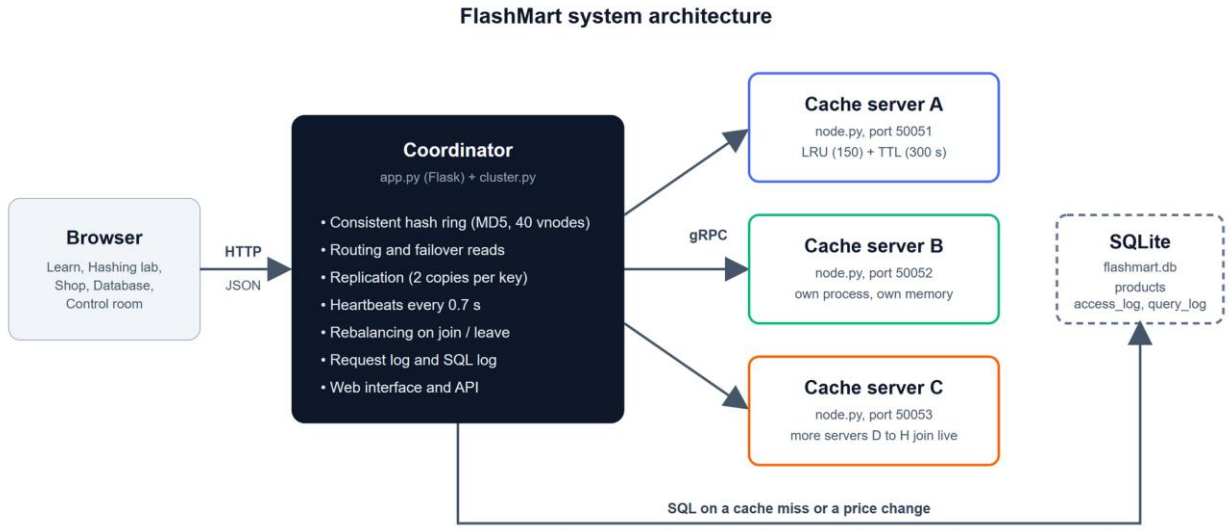


Figure 1: Architecture of FlashMart: the browser talks to the coordinator over HTTP; the coordinator calls the cache servers over gRPC and the SQLite database through SQL.

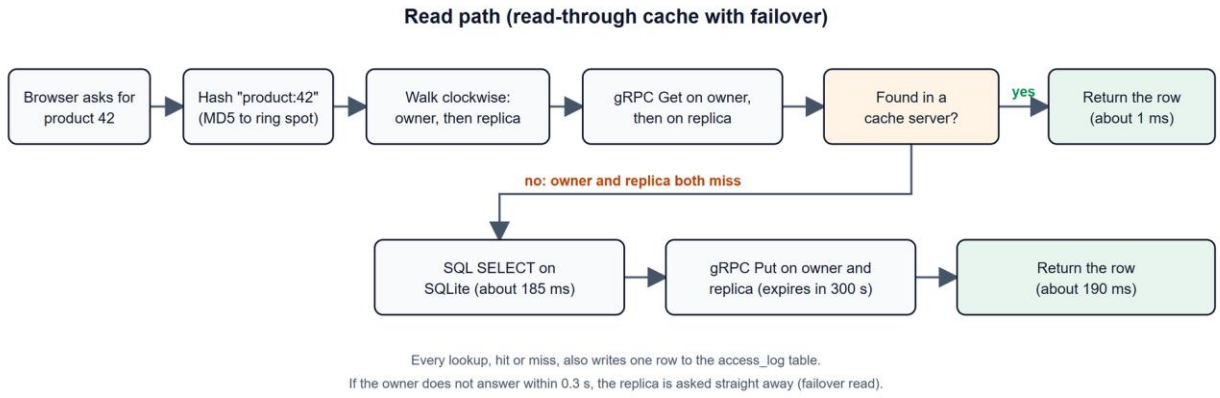


Figure 2: Read path of one product request, including the failover read and the access-log record.

4.3 Module Description

Module / Component	Responsibility	Input	Output
hashring.py	Consistent hash ring: virtual nodes, clockwise walk, owner and replica, hash details, `hash % N` baseline.	Key text, server names, virtual-node count.	Owner and replica list; every hop of the walk.
node.py (cache server)	In-memory key-value store with LRU eviction (150 entries) and TTL, served over gRPC.	gRPC calls: Get, Put, Delete, Keys, Ping, Stats, Clear.	Values, key lists, statistics.
cache.proto	The RPC contract shared by the coordinator and the servers.	Message definitions.	Generated client and server code.
cluster.py (coordinator)	Membership, routing, replication, failover reads, heartbeats, rebalancing, preview and trace functions.	Product requests; join, kill and reset commands.	Product rows, metrics, cluster state, event log.
db.py	SQLite database, simulated query delay, access and query logging, read-only SQL console.	SQL for products; log records.	Rows, `access_log` and `query_log` tables.
app.py	Flask web server and JSON API; terminal log of every hit, miss and SQL statement.	HTTP requests from the browser.	Web pages and JSON.
Web interface	Learn, Hashing lab, Shop, Database, Control room and Split view tabs; ring drawn in SVG.	State from the API.	Visualisation and controls.
show_db.py	Reads the database file from outside the application.	Command-line option (--tail).	Tables, totals, latest requests.
tests/test_hashring.py	Automated checks of ring behaviour.	Generated key sets.	10 passing tests.

4.4 Data / Inputs / Dataset (if applicable)

The data are generated, not collected. `db.py` builds 200 products with a fixed random seed (7): an identifier, a name made from an adjective and a product type (for example “Aero Sneakers”), a category, a price between 9 and 399, and a stock count between 0 and 500. The cache key of product N is the text `product:N`. Flash-sale traffic chooses product identifiers with skewed popularity (`id = int(200 × r^2.5)` for a random number r between 0 and 1), so low-numbered products are requested far more often, as in a real sale. The database has three tables: `products` (the 200 rows), `access_log` (one row per cache lookup: time, key, hit, miss or bypass, server, milliseconds, failover flag) and `query_log` (one row per SQL statement). The database is rebuilt from the seed data every time the application starts. No external dataset, personal information or licensed data is used.

4.5 Algorithm / Design Decisions (if applicable)

Hashing. The position of a key is the first 8 bytes of its MD5 digest read as a 64-bit number; dividing by 2^{64} and multiplying by 360 gives the angle on the ring. Each server S is placed at 40 virtual-node points `MD5("S#i")` for i = 0 to 39, kept in a sorted list. **Lookup** finds the first point at or after the key's position (wrapping around) with binary search; its server is the **owner**. Walking on and skipping points of servers already chosen gives the **replica**, the next different server clockwise (replication factor 2).

Read path. Ask the owner; if it has no copy or does not answer within 0.3 s, ask the replica; if neither has the key, run `SELECT` on the database, then store the row on both servers with a 300 s TTL. Every lookup is written to `access_log`. **Write path.** A price change runs `UPDATE` on the database and then deletes the cached copy from every live server (cache-aside invalidation), so the next read loads the new price.

Failure detection and rebalancing. A heartbeat thread pings each server every 0.7 s with a 0.4 s timeout. The first missed ping marks the server `suspect`; the second removes it from the ring. Rebalancing then lists the keys on every live server and, for each key, copies it to any server in its new preference list that lacks it and deletes it from servers that should no longer hold it. For every membership change the coordinator also records how many of the 200 products changed owner under consistent hashing and how many would have changed under `hash % N`.

Why a central coordinator. One coordinator holding the ring keeps routing simple and lets the whole cluster be drawn and explained in one place. The cost is a single point of failure, discussed in Section 8.2.

5. Methodology and Implementation

5.1 Workflow / Development Method

1. Choose the topic and define measurable outcomes: latency, hit ratio, keys moved, failover behaviour.
2. Write the gRPC contract (`cache.proto`) and generate the client and server code.
3. Implement the consistent hash ring and test it with automated tests before using it in the cluster.
4. Implement the cache server (LRU, TTL, statistics) and run several copies as separate processes.
5. Implement the coordinator: routing, replication, failover reads, heartbeats and rebalancing.
6. Add the SQLite database with a simulated delay, then add read-through caching and invalidation on writes.
7. Build the web interface: ring drawing, shop, control room and guided demonstration.
8. Add the Hashing lab (step-by-step trace and join preview) and the Database tab.
9. Add backend visibility: terminal log, `access_log` and `query_log` tables, `show_db.py`, SQL console.
10. Measure the system, capture screenshots and write the report.

5.2 Technologies, Tools and Resources

Category	Technology / Tool	Purpose / Justification
Programming language / framework	Python 3.14, Flask 3.1	Coordinator logic, cache servers and the web API.
RPC	gRPC 1.85 and Protocol Buffers	Calls between the coordinator and the cache servers.
Database / storage	SQLite 3 (file `flashmart.db`)	Real source of truth and request logs; no database server to install.
Front end	HTML, CSS, JavaScript, SVG	Single-page interface and the ring drawing, with no framework.
Development / testing tools	pytest, `show_db.py`, browser developer tools	Ring tests, backend inspection and manual end-to-end checks.

Category	Technology / Tool	Purpose / Justification
Report tooling	Playwright with Microsoft Edge; Matplotlib	Used only to capture screenshots and draw the charts in this report.
Hardware / cloud / other resources	Student laptop (Windows 11)	No cloud service, paid API or special hardware is needed.

5.3 Implementation Details

The code base is about 2,900 hand-written lines: 1,143 lines of Python for the backend (`app.py` 181, `cluster.py` 488, `db.py` 170, `hashring.py` 118, `node.py` 112, `show_db.py` 74), 87 lines of tests, 1,219 lines of web interface and 50 lines of protocol definition. Each cache server is started with `python node.py --name A --port 50051` and keeps an ordered dictionary protected by a lock; when more than 150 entries are stored the least recently used one is dropped, and entries expire after the TTL given on `Put`. The gRPC server uses 16 worker threads. The coordinator starts servers A, B and C as subprocesses on ports 50051 to 50053 and can add servers D to H while running. gRPC channels use a short reconnect back-off (200 to 500 ms) so a restarted server is reachable quickly.

The database module wraps SQLite in WAL mode. Every query sleeps for 180 ms plus 4 ms for each query in flight, writes an entry to `query_log`, and prints a `SQL` line to the terminal; every cache lookup writes to `access_log` and prints a `HIT`, `MISS` or `BYPASS` line. The SQL console opens a separate read-only connection and accepts only `SELECT` or `WITH` statements. The web interface polls the state every 0.7 s and redraws the ring; a coloured band shows which server owns each stretch of the ring, outer ticks are virtual nodes and inner dots are the 200 products, filled when cached.

5.4 User Interface / Prototype / Output

The prototype is a single web page with six tabs. The screenshots below were captured from the running system. Learn explains the idea and lets the user look up any key; Hashing lab shows consistent hashing step by step; Shop is the customer view; Database shows the SQLite table and the requests it served; Control room is the engineer view; Split view shows Shop and Control room together for presenting.

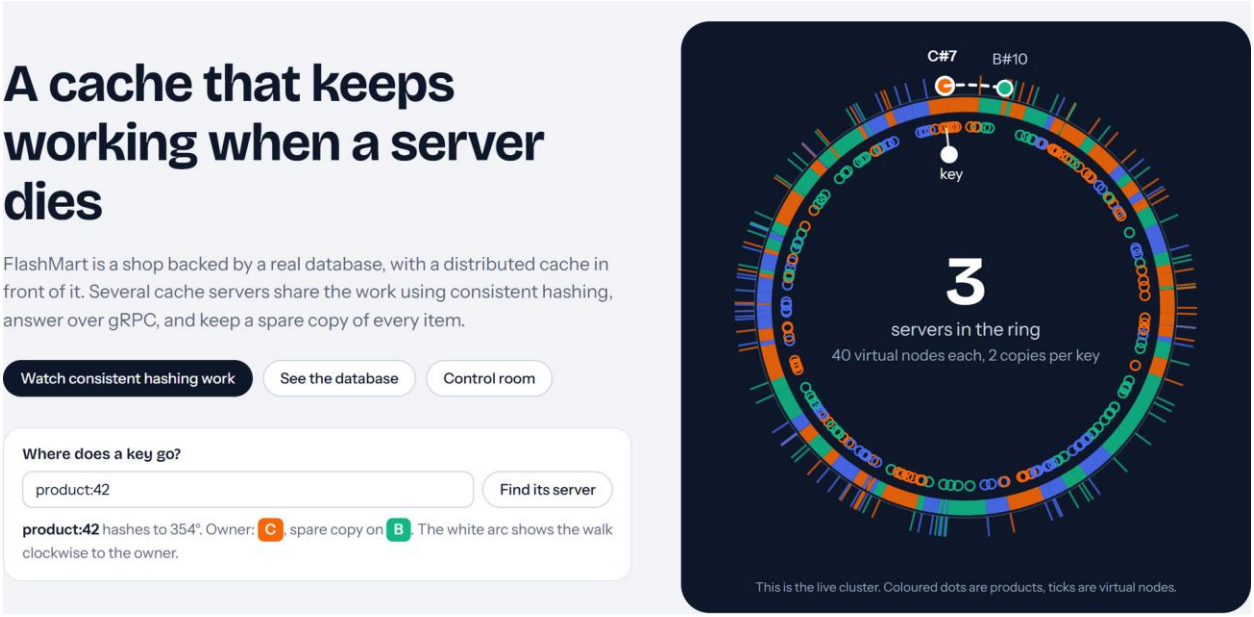


Figure 3: Learn screen: overview, a lookup box ("Where does a key go?") and the live ring. The white arc is the clockwise walk from the key to its owner.

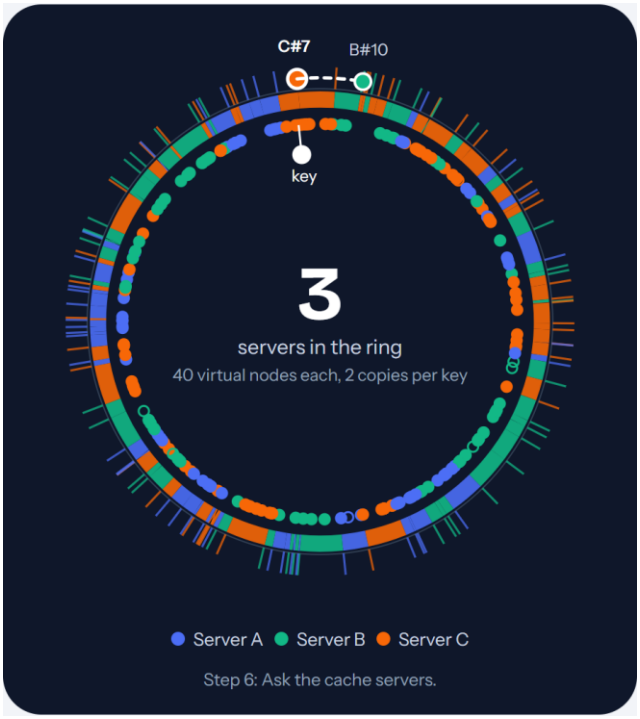


Figure 4: Hashing lab ring at step 6 for product:42. The white dot is the key (353.77°); the owner is virtual node C#7 and the dashed arc continues to B#10, which holds the replica.

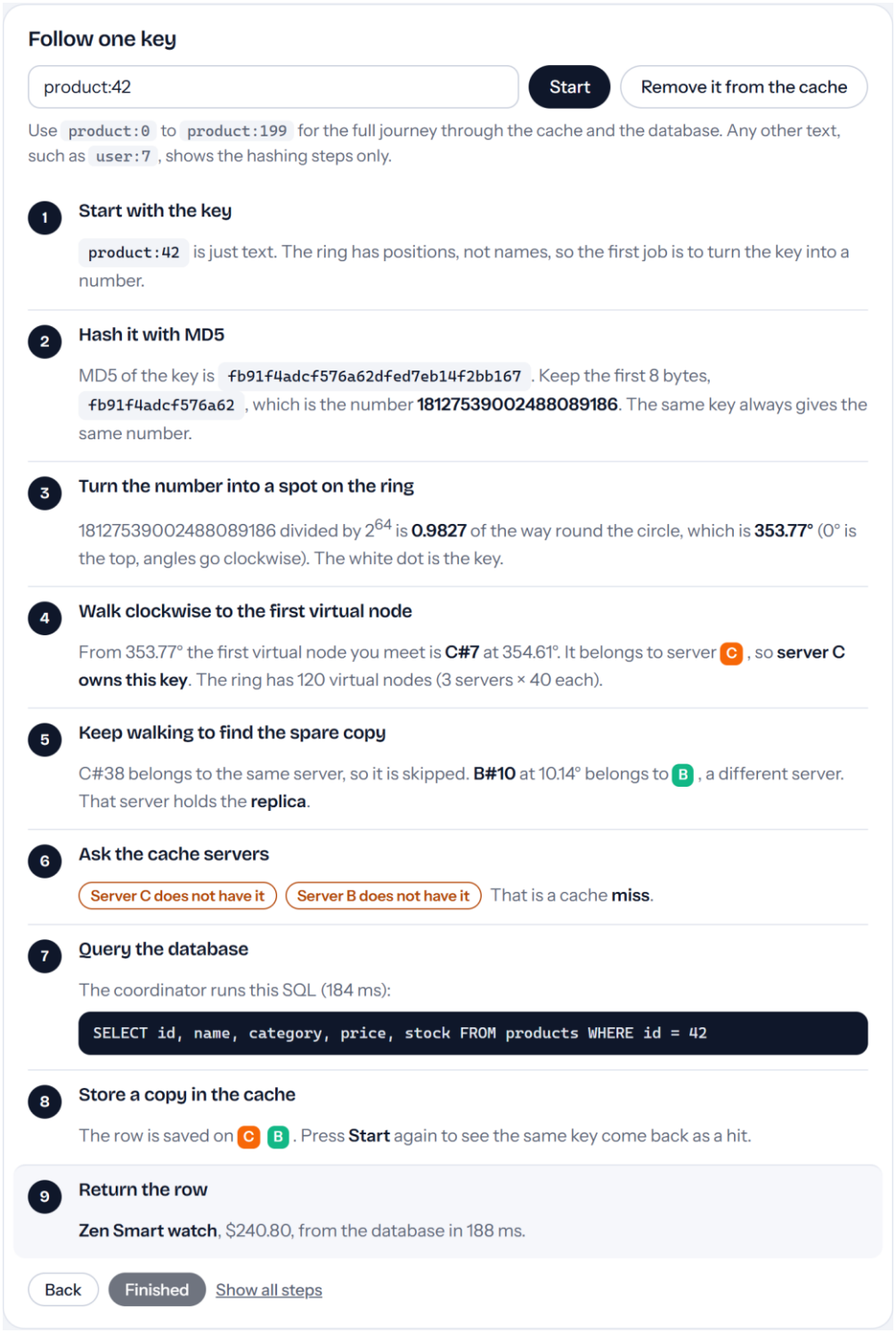


Figure 5: Hashing lab, the nine steps for product:42 on a cache miss: key, MD5 hash, ring position, clockwise walk to the owner, search for the replica, cache lookup, SQL query, store on both servers, return the row.

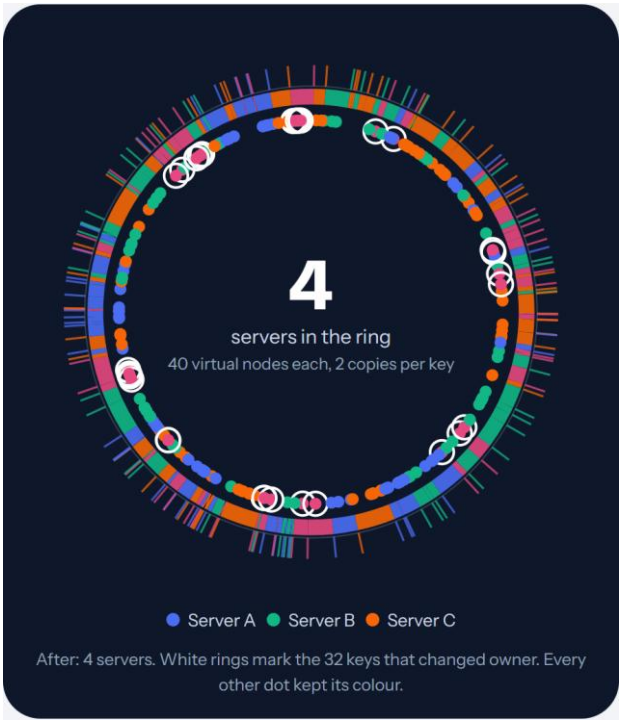




Figure 6: Hashing lab, preview of adding a fourth server (real cluster unchanged). Top: ring after the change; white rings mark the 32 keys that change owner. Bottom: each square is one product; consistent hashing moves 32 of 200 (green), plain hash % N would move 151 (red).

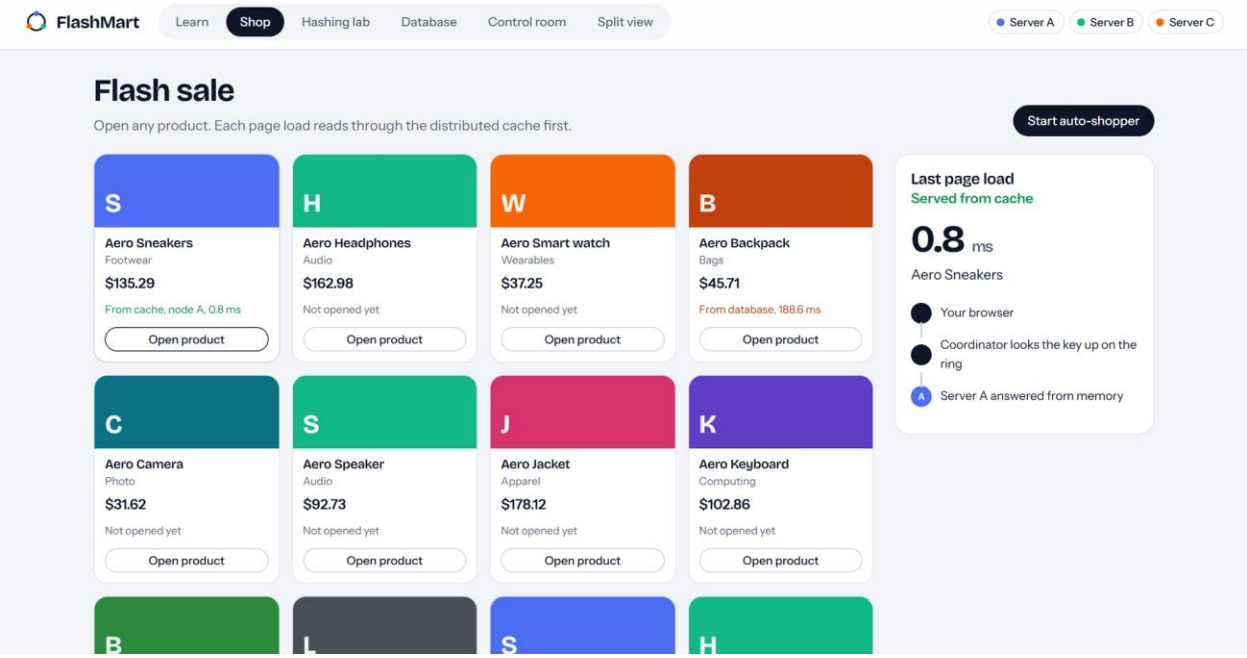


Figure 7: Shop screen: each card reports where it was served from. The "Last page load" panel lists each hop of the request (here, Server A answered in 0.8 ms).

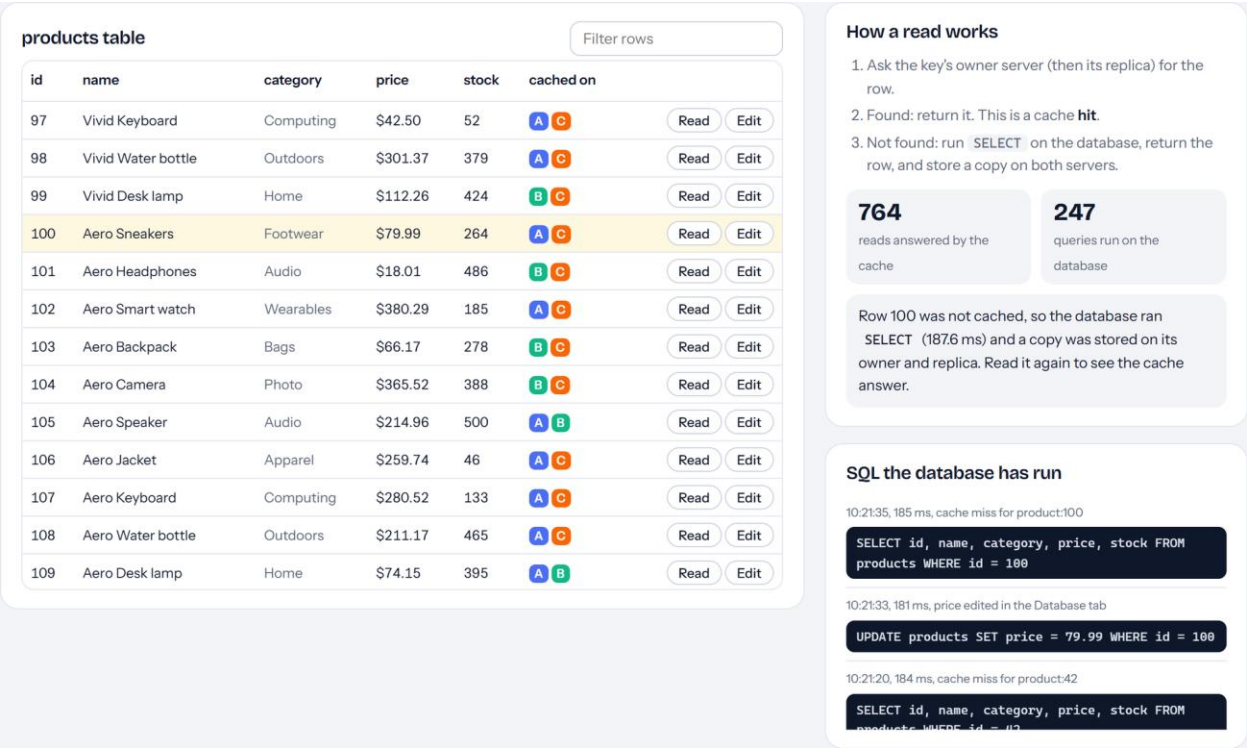


Figure 8: Database tab: the SQLite products table with the servers that hold a cached copy of each row, how a read works, and the SQL the database has run (the highlighted row 100 was read, edited with an UPDATE and read again).

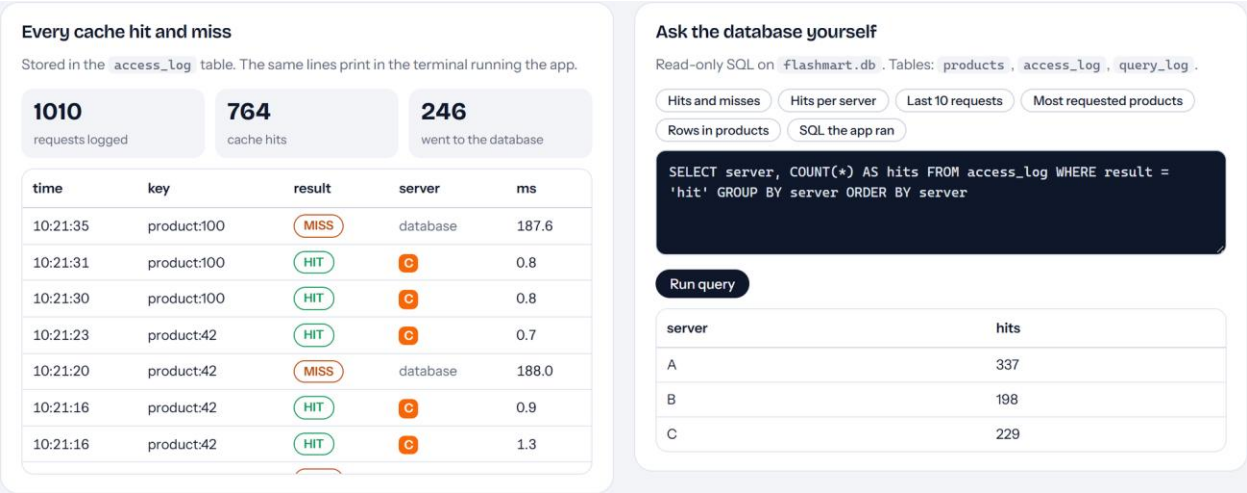


Figure 9: Database tab: every cache hit and miss from the access_log table, and the read-only SQL console showing hits per server.



Figure 10: Control room after the first demonstration step (flash sale of 1000 requests): live ring, hit ratio, response time, database queries and the guided demo steps.

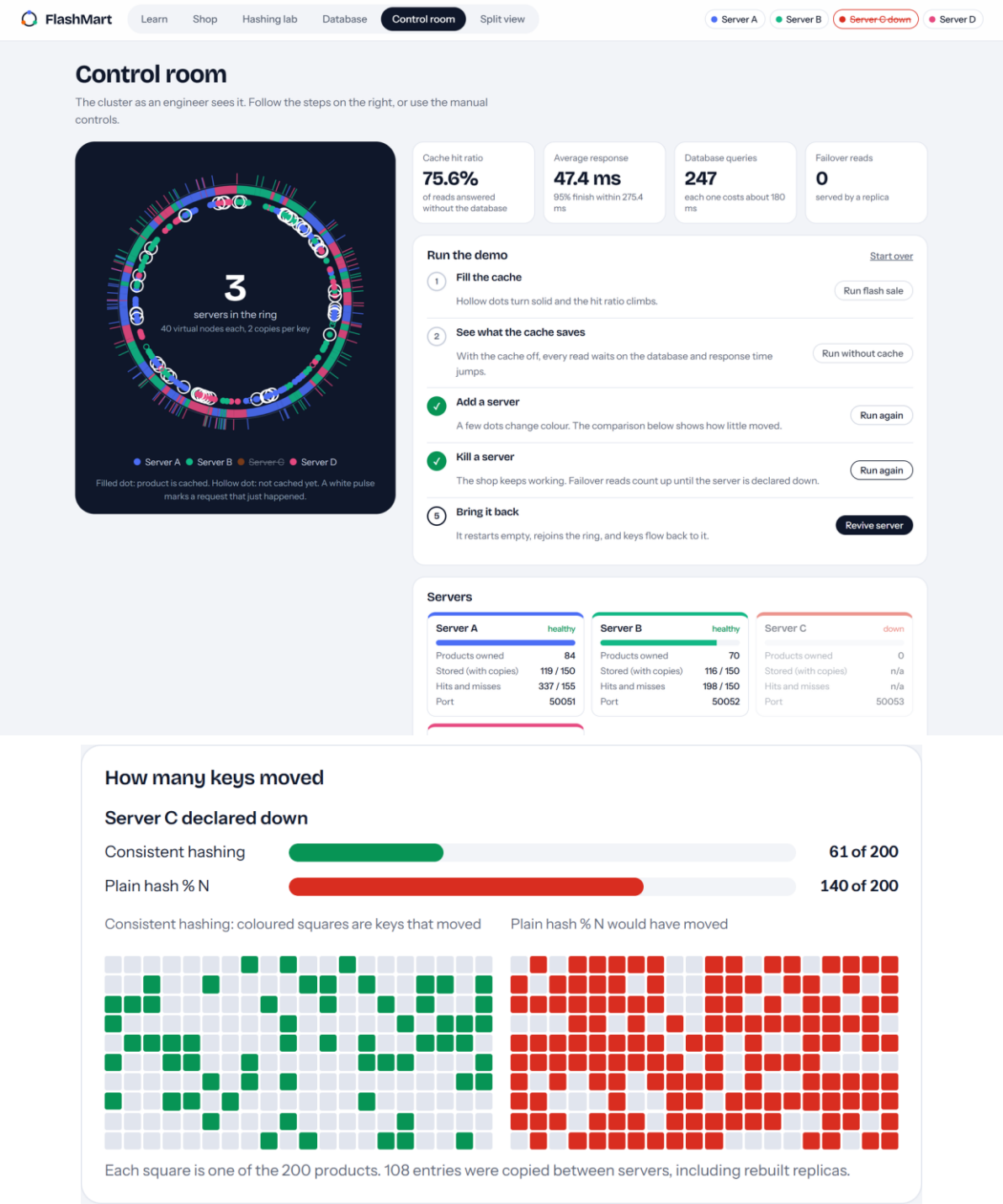


Figure 11: Control room after a server was added and then killed. Server C is shown as down, white rings mark keys that moved, and the comparison panel shows 61 of 200 keys moved against 140 for plain hash % N.

Terminal running app.py (backend log, lines copied from the real run)

```
Backend log: HIT / MISS / BYPASS per request, SQL per database query, EVENT for cluster changes
10:20:57  EVENT  Cluster started with servers A, B, C
10:21:07  SQL    SELECT id, name, category, price, stock FROM products WHERE id = 0  (185 ms)
10:21:07  MISS   product:0     database     189.2 ms
10:21:08  SQL    SELECT id, name, category, price, stock FROM products WHERE id = 3  (184 ms)
10:21:08  MISS   product:3     database     188.6 ms
10:21:09  HIT    product:0     server A       0.8 ms
10:21:11  EVENT  Flash sale started: 1000 requests from 24 concurrent users
10:21:11  HIT    product:0     server A       3.4 ms
   … (flash-sale request lines omitted) …
10:21:14  MISS   product:52    database     262.4 ms
10:21:14  EVENT  Flash sale finished
10:21:16  HIT    product:42    server C       1.3 ms
10:21:16  HIT    product:42    server C       0.9 ms
10:21:19  EVENT  product:42 evicted from the cache (was on B, C)
10:21:20  SQL    SELECT id, name, category, price, stock FROM products WHERE id = 42  (184 ms)
10:21:20  MISS   product:42    database     188.0 ms
10:21:23  HIT    product:42    server C       0.7 ms
10:21:30  HIT    product:100   server C       0.8 ms
10:21:31  HIT    product:100   server C       0.8 ms
10:21:33  SQL    UPDATE products SET price = 79.99 WHERE id = 100  (181 ms)
10:21:33  EVENT  Price of product 100 changed in the database; cached copies on A, C deleted
10:21:35  SQL    SELECT id, name, category, price, stock FROM products WHERE id = 100  (185 ms)
10:21:35  MISS   product:100   database     187.6 ms
10:21:42  EVENT  Server D joined: consistent hashing moved 32/200 keys (plain hash % N would move 151/200)
10:21:45  EVENT  Server C process killed
10:21:46  EVENT  Server C missed a heartbeat
10:21:47  EVENT  Server C declared down: consistent hashing moved 61/200 keys (plain hash % N would move 140/200)
10:21:53  EVENT  Server C joined: consistent hashing moved 61/200 keys (plain hash % N would move 140/200)
```

Figure 12: Backend terminal of the running application. Lines are copied from the real run log (flash-sale lines omitted): MISS with its SQL SELECT, then HIT, the UPDATE and cache invalidation, a server joining, being killed and declared down.

python show_db.py (reads flashmart.db directly)

```
Database file: C:\Ishan\DC_microproject\flashmart.db  (745472 bytes)
Tables and row counts
  table       rows
  ----------  ----
  products    200
  access_log  1020
  query_log   247
products (first 8 rows)
  id  name             category   price   stock
  --  ---------------  ---------  ------  -----
  0   Aero Sneakers    Footwear   135.29  77
  1   Aero Headphones  Audio      162.98  24
  2   Aero Smart watch Wearables  37.25   274
  3   Aero Backpack    Bags       45.71   298
  4   Aero Camera      Photo      31.62   259
  5   Aero Speaker     Audio      92.73   44
  6   Aero Jacket      Apparel    178.12  35
  7   Aero Keyboard    Computing  102.86  282
Cache results (access_log)
  result  requests  avg_ms
  ------  --------  ------
  hit     774       0.6
  miss    246       271.5
Cache hits per server
  server  hits
  ------  ----
  A       340
  B       204
  C       230
Latest 10 requests
  time                key         result  server  ms
  ------------------  ----------  ------  ------  ---
  2026-11-06 10:22:03  product:8   hit     B       0.7
  2026-11-06 10:22:03  product:28  hit     B       0.7
  2026-11-06 10:22:02  product:5   hit     B       1.0
  2026-11-06 10:22:02  product:7   hit     C       0.7
  2026-11-06 10:22:01  product:16  hit     B       0.8
  2026-11-06 10:22:01  product:29  hit     B       0.8
  2026-11-06 10:22:00  product:27  hit     A       0.7
  2026-11-06 10:22:00  product:16  hit     B       0.8
  2026-11-06 10:21:59  product:34  hit     A       0.7
  2026-11-06 10:21:59  product:2   hit     A       0.9
Latest 5 SQL statements the application ran (query_log)
  time                ms     sql
  ------------------  -----  ------------------------------------------------------------------------
  2026-11-06 10:21:35  185.0  SELECT id, name, category, price, stock FROM products WHERE id = 100
  2026-11-06 10:21:33  181.0  UPDATE products SET price = 79.99 WHERE id = 100
  2026-11-06 10:21:20  184.0  SELECT id, name, category, price, stock FROM products WHERE id = 42
  2026-11-06 10:21:14  260.0  SELECT id, name, category, price, stock FROM products WHERE id = 52
  2026-11-06 10:21:14  272.0  SELECT id, name, category, price, stock FROM products WHERE id = 58
```

Figure 13: Output of show_db.py, which opens flashmart.db directly: row counts, hits and misses, hits per server, latest requests and the latest SQL statements.

5.5 Team Contribution (for group projects)

Team Member	Assigned Responsibility	Work Completed / Evidence
[Student name 1 (Class – Roll)]	[e.g. Hash ring, replication, failover and rebalancing]	[Describe the work actually done, e.g. hashring.py, cluster.py, ring tests]
[Student name 2 (Class – Roll)]	[e.g. Database, logging, gRPC cache servers and testing]	[Describe the work actually done, e.g. db.py, node.py, show_db.py, benchmarks]
[Student name 3 (Class – Roll)]	[e.g. Web interface, demonstration and documentation]	[Describe the work actually done, e.g. web pages, screenshots, this report]

6. Testing, Evaluation and Results

6.1 Test Plan and Test Cases

Two kinds of checks were used: ten automated unit tests of the hash ring (pytest), and scripted end-to-end runs against the live system through its HTTP API and browser interface. The coordinator, the gRPC integration and the web interface have no automated tests. Multi-machine operation was not tested and is marked pending.

Test ID	Test Input / Scenario	Expected Result	Observed Result	Status
TC-01	Run the 10 pytest tests on the hash ring.	Deterministic placement; distinct replicas; about 1/N keys move; vnodes improve balance.	10 passed.	PASS
TC-02	Read 30 products twice through the cache.	First read from the database, second from the cache.	Mean 188.2 ms (miss) then 0.88 ms (hit).	PASS
TC-03	Flash sale: 1000 requests, 24 users, cache on.	Most reads answered by the cache.	756 hits, 244 misses (75.6%); 244 database queries.	PASS
TC-04	Flash sale: 300 requests, 24 users, cache off.	Every request reaches the database and is slower.	300 of 300 reached the database; mean 272.8 ms.	PASS
TC-05	Add a server to the running cluster.	Only about 1/N of keys move; shop keeps working.	3→4 servers: 32 of 200 moved (modulo: 151). 4→5: 43 of 200 (modulo: 159); took 0.66 s.	PASS
TC-06	Kill the busiest server while reads continue.	No failed requests; failure detected; replicas rebuilt.	0 errors in 507 and 339 reads; down in about 2.1 s; 20 failover reads in each run.	PASS
TC-07	Restart the killed server.	Server rejoins the ring.	Rejoined in 0.60 s; keys flowed back.	PASS
TC-08	Edit a price with the Database tab.	UPDATE runs; cached copies deleted; next read shows the new price.	UPDATE logged; copies on A and C deleted; next read from the database showed the new price.	PASS
TC-09	Send DROP TABLE to the SQL console.	Query rejected; data unchanged.	Rejected: “Only SELECT queries are allowed here.”	PASS
TC-10	Check backend visibility after a run.	Every lookup logged in the terminal and in `access_log`.	Terminal showed HIT/MISS/SQL lines; show_db.py reported 774 hits and 246 misses in `access_log` after one run.	PASS
TC-11	Run cache servers on a second computer and attach them.	Servers join and fail over as locally.	Option implemented (`--attach`) but not exercised on a real network.	PENDING

6.2 Evaluation Metrics

The evaluation uses response time, hit ratio, database load, placement stability, load balance and fault behaviour. Hit ratio is hits divided by all lookups. Response time is measured at the coordinator from receiving the request to returning the row. Keys moved is the number of the 200 products whose owner changes when the set of servers changes; the same count is computed for `hash(key) % N` on the same keys for comparison (the comparison is calculated, not run on a second cache). Balance is the standard deviation of keys per server as a percentage of the mean (lower is better).

Metric	Measurement Method / Formula	Observed Value	Interpretation
Hit latency vs miss latency	Mean time of the second and first read of 30 products	0.88 ms vs 188.2 ms	A hit is about 213 times faster.
Cache hit ratio	hits / (hits + misses) × 100	75.6% (1000 requests)	Three of four reads avoided the database.
Average response time	Mean per-request time	41.6 ms with cache; 272.8 ms without	About 6.6 times lower with the cache.
95th percentile response	95th percentile of per-request times	275.3 ms with cache; 276.7 ms without	With the cache the slowest 5% are first-time misses under load.
Database queries	Count of SELECTs	244 of 1000 vs 300 of 300	Database load fell to 24% of requests.
Keys moved on join	Owner changed / 200	32 (16%) vs 151 (75.5%) for 3→4 servers	Consistent hashing moves a fraction close to 1/N.
Load balance	Std. deviation of keys per server / mean	13.3% at 40 vnodes; 65.0% at 1	More virtual nodes give a much more even split.
Failure detection time	Time from process kill to status	Suspect after about 0.8–0.9 s; down after about 2.1 s	Matches the 0.7 s heartbeat with two misses.
Failed requests during failure	HTTP errors while a server is killed	0 of 507 and 0 of 339	Replication hides a single-server failure.
Rebalance duration	Time to add or revive a server	0.66 s and 0.60 s	Short pause while keys are copied.

6.3 Results and Discussion

All measurements below were taken on one laptop with three cache servers (A, B, C) unless stated, 40 virtual nodes per server, replication factor 2 and 24 concurrent simulated users. Values differ a little from run to run; the screenshots come from separate demonstration runs and can differ slightly from these benchmark tables (for example 75.1% against 75.6% hit ratio).

Cache benefit. Reading each of 30 products twice, the first read (database) averaged 188.2 ms (range 187.1–189.6 ms) and the second (cache) 0.88 ms (0.60–1.20 ms), a factor of about 213.

Condition	Requests	Hits	Misses	Hit ratio	Avg (ms)	p95 (ms)	DB queries	Time
Cache on	1000	756	244	75.6%	41.6	275.3	244	3.1 s
Cache off	300	0	300	0%	272.8	276.7	300	3.8 s

With the cache the system handled about 320 requests per second against about 79 without it. The 272.8 ms average with the cache off is close to 180 ms + 4 ms × 24 concurrent queries, which is the overload model in the database module. There were 244 misses for only 200 products because several users can ask for the same uncached product at the same moment and each one queries the database; the system does not merge such requests.

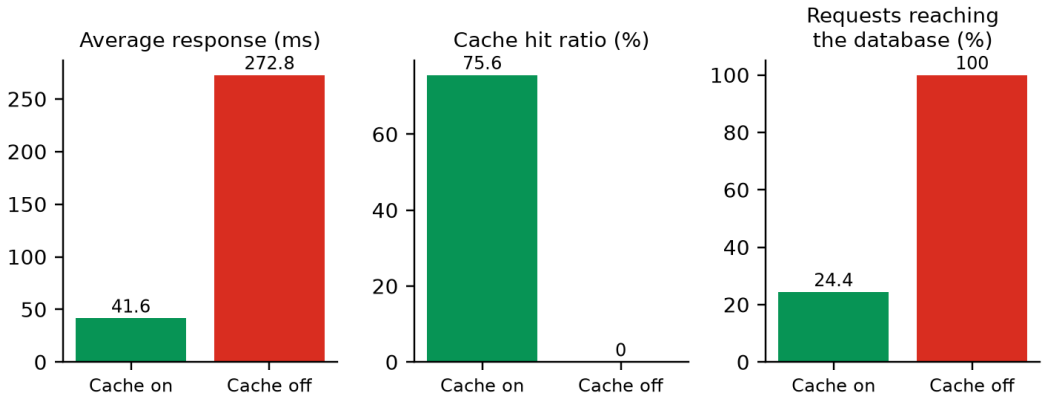


Figure 14: Effect of the cache in the flash-sale test: average response time, hit ratio and share of requests that reached the database.

Placement stability. The table and chart compare keys moved when one server is added, over 2,000 keys with 40 virtual nodes. Removing a server behaves the same way (for example 4→3 servers: 22.0% against 74.3%).

Servers (before → after)	Ideal 1/(N+1)	Consistent hashing	Plain hash % N
2 → 3	33.3%	36.8%	65.5%
3 → 4	25.0%	22.0%	74.3%
4 → 5	20.0%	20.6%	80.7%
5 → 6	16.7%	14.1%	84.2%
6 → 7	14.3%	12.3%	84.8%
7 → 8	12.5%	11.9%	88.5%

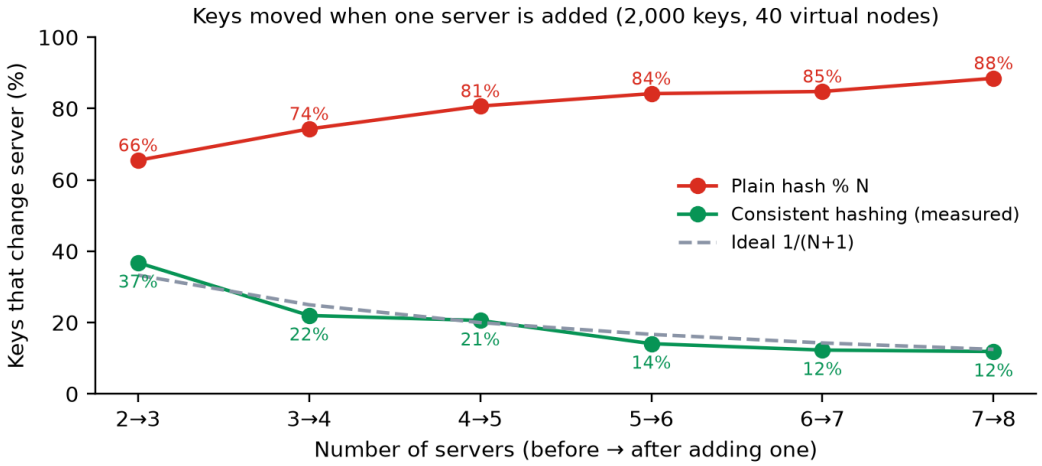


Figure 15: Percentage of keys that change server when one server is added: consistent hashing stays close to the ideal 1/(N+1) while hash % N approaches 100%.

On the 200 shop products, going from three to four servers moved 32 keys (16%) with consistent hashing against 151 (75.5%) for `hash % N`; in the live cluster, adding a fifth server moved 43 (21.5%) against 159 (79.5%), copying 94 cache entries to their new owners and deleting 94 from old owners in 0.66 s.

Load balance. With one virtual node per server the split of 2,000 keys over three servers was very uneven (79, 1112 and 809). More virtual nodes narrow it; the gain levels off at about 100.

Virtual nodes	Server A	Server B	Server C	Max / min	Std. dev. (% of mean)
1	79	1112	809	14.08	65.0
5	447	725	828	1.85	24.1
10	513	898	589	1.75	25.0
40 (default)	541	724	735	1.36	13.3
100	639	692	669	1.08	3.3
200	635	713	652	1.12	5.0

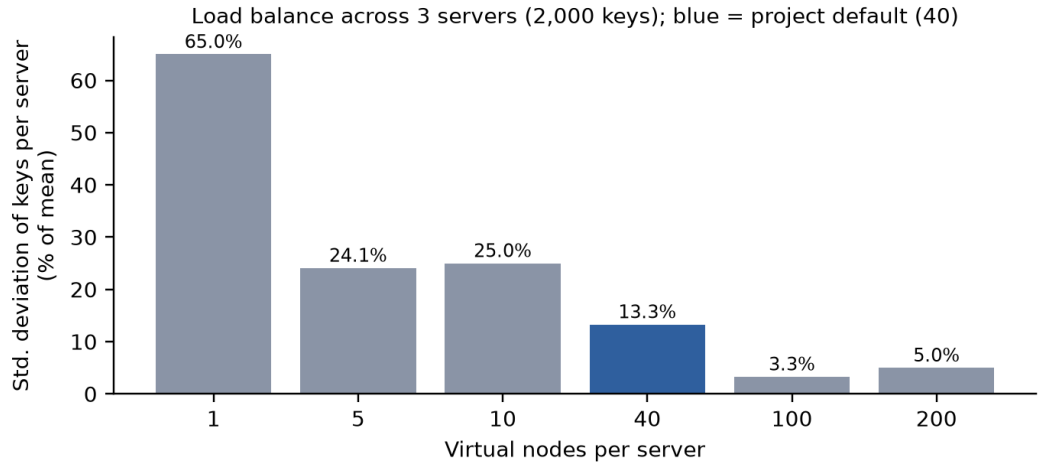


Figure 16: Spread of keys over three servers against the number of virtual nodes per server (lower is more even).

Fault tolerance. Four reader threads requested products continuously while the busiest server's process was killed. Both runs show no failed request.

Measurement	Run 1 (server C killed)	Run 2 (server B killed)
Server status after the kill	suspect at about 0.9 s; down at about 2.06 s	suspect at about 0.8 s; down at about 2.07 s
Reads issued after the kill	507	339
Reads that returned an error	0	0
Reads answered by a replica (failover)	20	20
Slowest read after the kill	409 ms (client side)	about 315 ms (server side)
Keys re-assigned when declared down	61 of 200 (hash % N: 140)	44 of 200 (hash % N: 167)
Cache entries copied to rebuild replicas	107	67

The only visible cost of the failure was a short spike to about 300–400 ms for reads whose owner had just died, caused by the 0.3 s timeout before the replica is asked. Within about two seconds the server was removed from the ring and its keys were assigned to the surviving servers.

Database and invalidation. Editing the price of product 100 ran `UPDATE products SET price = 79.99 WHERE id = 100` (181 ms) and deleted the cached copies on servers A and C. The next read of the product missed, ran a `SELECT` (185 ms) and returned the new price, which was then cached again. The `access_log` table recorded every lookup of the run (1,020 rows: 774 hits, 246 misses), and `query_log` recorded 247 statements.

6.4 Validation / Demonstration Evidence

Demonstration sequence: start the application (it launches servers A, B and C); open Learn and look up a key; in the Hashing lab follow `product:42` through all nine steps and preview adding a server; in Shop open one product twice (database, then cache); in Database read a row twice, edit its price and read again while the terminal shows the matching `MISS`, `SQL`, `HIT` and `UPDATE` lines; run `python show_db.py` to show the data in the database file; in Control room run the flash sale, add a server and kill a server; finally revive it. The corresponding evidence is Figures 3 to 13. The application runs at `http://localhost:8000/` on the demonstration laptop; the address is not a public deployment.

7. Sustainability and SDG Alignment (Mandatory for Every Project)

7.1 Sustainability Assessment

Dimension	Questions to Consider	Project-Specific Assessment / Evidence
Environmental	Energy and compute use; paper/material use; e-waste; resource efficiency; emissions.	Indirect, modest positive potential: in the test the cache answered 75.6% of reads, so the database did a quarter of the work, which is the efficiency idea behind real caches. The project runs on an existing laptop with synthetic data. Energy use was not measured.
Social	Accessibility and inclusion; user safety; privacy; fairness; digital divide; community benefit.	Educational value for students learning distributed systems. No personal data is collected. The interface needs a browser and Python knowledge to run; fonts load from the internet when available and fall back to system fonts offline.

Dimension	Questions to Consider	Project-Specific Assessment / Evidence
Economic	Development and operating cost; maintenance; affordability; productivity; long-term viability.	Built entirely with free, open-source software on an existing laptop; no paid service or database server. In real systems, caching reduces the database capacity that must be bought, but this was not measured here.

7.2 UN Sustainable Development Goals (SDGs)

SDG 4 – Quality Education: the project is a hands-on, reproducible learning artefact for partitioning, replication, failure handling and performance measurement in distributed systems. SDG 9 – Industry, Innovation and Infrastructure is an indirect, conceptual link, because distributed caching is a common technique for building efficient digital infrastructure; no real infrastructure deployment or measurable industrial impact is claimed.

SDG No. and Name	Project Feature / Activity Linked to SDG	Expected or Observed Contribution	Evidence / Indicator
SDG 4: Quality Education	Hashing lab, guided demonstration, backend inspection tools and documentation.	Indirect: supports practical understanding of distributed systems.	Working demonstration, EXPLAINED.md and this report; learning outcome not formally surveyed.
SDG 9: Industry, Innovation and Infrastructure	Distributed cache with replication, failover and rebalancing in front of a database.	Conceptual: shows how infrastructure can stay fast and available as servers change.	Measured: 75.6% of reads served from cache; no failed requests when a server was killed. No production impact is claimed.

7.3 Sustainability Measures and Trade-offs

The project uses synthetic data and a fixed random seed so experiments are repeatable without collecting data. The cache servers are lightweight processes that stop when the application exits. The trade-off of caching is memory: replication stores every key twice, so memory use doubles in exchange for fault tolerance, and more servers use more compute. For a real deployment, memory, energy and over-provisioning should be monitored.

7.4 Sustainability Indicator / Follow-up

Proposed indicators, not measured in this report: CPU and memory use of the cache servers during a flash sale; database queries avoided per 1,000 requests (measured here as 756 of 1,000); memory per cached entry; and energy use of the laptop with and without the cache over repeated runs.

8. Feasibility, Limitations and Risk Considerations

8.1 Feasibility

The project is technically feasible with Python, gRPC and SQLite on one laptop; no special hardware is needed. The ring, the cache servers and the coordinator are small and modular, so the work fits a

laboratory micro-project. Operational feasibility is limited by the single machine and the single coordinator. The solution is low-cost because every tool is free and open source.

Item / Resource	Estimated Cost (₹)	Basis / Notes
Hardware / hosting / cloud	₹0 (estimated)	Uses an existing laptop; no cloud or hosting.
Software / API / dataset	₹0	Python, Flask, gRPC, SQLite, pytest, Matplotlib and Playwright are open source; the data are synthetic.
Other resources	₹0	No paid API, external dataset or dedicated server.
Total estimated cost	₹0	Direct project cost; excludes the existing device, electricity and internet.

8.2 Limitations

- **Single coordinator.** It holds the ring and routes every request; if it stops, the system stops. A production design would replicate it or let clients hold the ring.
- **Lock during rebalancing.** Membership changes hold the coordinator's lock while keys are copied, so reads that need ring information briefly wait (about 0.6 s for a join in these tests).
- **Simple failure detector.** Fixed-interval heartbeats with a two-miss threshold; no gossip, adaptive thresholds or protection against false alarms and network partitions.
- **Eventual, best-effort consistency.** No quorum, versioning or read repair. Copies made during rebalancing get the default 300 s TTL rather than their remaining lifetime.
- **Cache-aside race.** A slow read that missed can re-cache an old value just after a price update has deleted the cached copy; this is not handled.
- **No request coalescing.** Simultaneous misses for the same product each query the database (244 misses for 200 products).
- **No persistence.** A killed cache server loses its data; correctness holds because the database is the source of truth and a replica exists.
- **One machine only.** Network delay, packet loss and partitions were not tested; the multi-machine option was not exercised.
- **Simulated database delay.** The database is a local SQLite file with an artificial delay and is not itself distributed.
- **Small data set and development server.** 200 products and a 150-entry capacity mean LRU eviction rarely occurs; the Flask development server is used.
- **Calculated comparison.** The `hash % N` figures are computed on the same keys; no second cache using modulo placement was run.
- **Test coverage.** Only the hash ring has automated tests; the rest was verified by scripted runs and manual checks.

8.3 Risks, Ethics, Privacy and Security

The system uses synthetic data and processes no personal information. gRPC channels are unencrypted and unauthenticated, and the web server listens on all network interfaces so that others on the same network can open it; it should be run only on a trusted network, or bound to 127.0.0.1, and never exposed to the internet. The SQL console is read-only and accepts only SELECT statements, but it is not password protected and exposes the logs. Failures and load are simulated, so results should be read as illustrations

of behaviour, not as capacity claims. No AI model is used and no decision about people is made. A production version would need TLS, authentication, input validation, rate limiting and monitoring.

9. Conclusion and Future Scope

9.1 Summary

The project delivers a working distributed cache with consistent hashing, demonstrated through the FlashMart shop on a real SQLite database. Cache servers run as separate gRPC processes; keys are placed on a ring with virtual nodes, stored on two servers, and rebalanced automatically when servers join, leave or fail. In the tests a cache hit was about 213 times faster than a database read, the cache answered 75.6% of flash-sale requests and reduced the mean response time from 272.8 ms to 41.6 ms, adding a server moved 16% of keys against 75.5% for modulo placement, and killing a server caused no failed requests. The web interface, the Hashing lab, the backend log, the database tables and the SQL console make each step visible. The system is an educational prototype on one machine, not a production cluster; its main limits are the single coordinator, simple failure detection, eventual consistency and the lack of multi-machine tests.

9.2 Future Scope

1. Replicate the coordinator, or move the ring into the client, to remove the single point of failure.
2. Add quorum reads and writes with versioning and read repair for stronger consistency.
3. Use gossip or adaptive (phi-accrual) failure detection and test with injected network faults.
4. Merge simultaneous misses for the same key (request coalescing) and handle very popular keys.
5. Warm the cache of a joining server and avoid pausing reads during rebalancing.
6. Add persistence (snapshots) and TLS with authentication to the cache servers.
7. Run the cache servers on separate machines or containers and repeat the measurements.
8. Add automated integration tests for the coordinator and the web interface and use a production web server.

References

References are listed in IEEE style. The algorithm descriptions in this report are project-level summaries of the cited ideas.

[1] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine and D. Lewin, “Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web,” in Proc. 29th Annual ACM Symposium on Theory of Computing (STOC), 1997, pp. 654–663.

[2] G. DeCandia et al., “Dynamo: Amazon’s highly available key-value store,” in Proc. 21st ACM Symposium on Operating Systems Principles (SOSP), 2007, pp. 205–220.

[3] R. Nishtala et al., “Scaling Memcache at Facebook,” in Proc. 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2013, pp. 385–398.

[4] gRPC Authors, “gRPC Documentation.” https://grpc.io/docs/

[5] SQLite Consortium, “SQLite Documentation.” https://www.sqlite.org/docs.html

[6] Pallets Projects, “Flask Documentation.” https://flask.palletsprojects.com/

[7] Python Software Foundation, “Python 3 Documentation.” https://docs.python.org/3/

Appendices

Appendix A: Additional screenshots / outputs

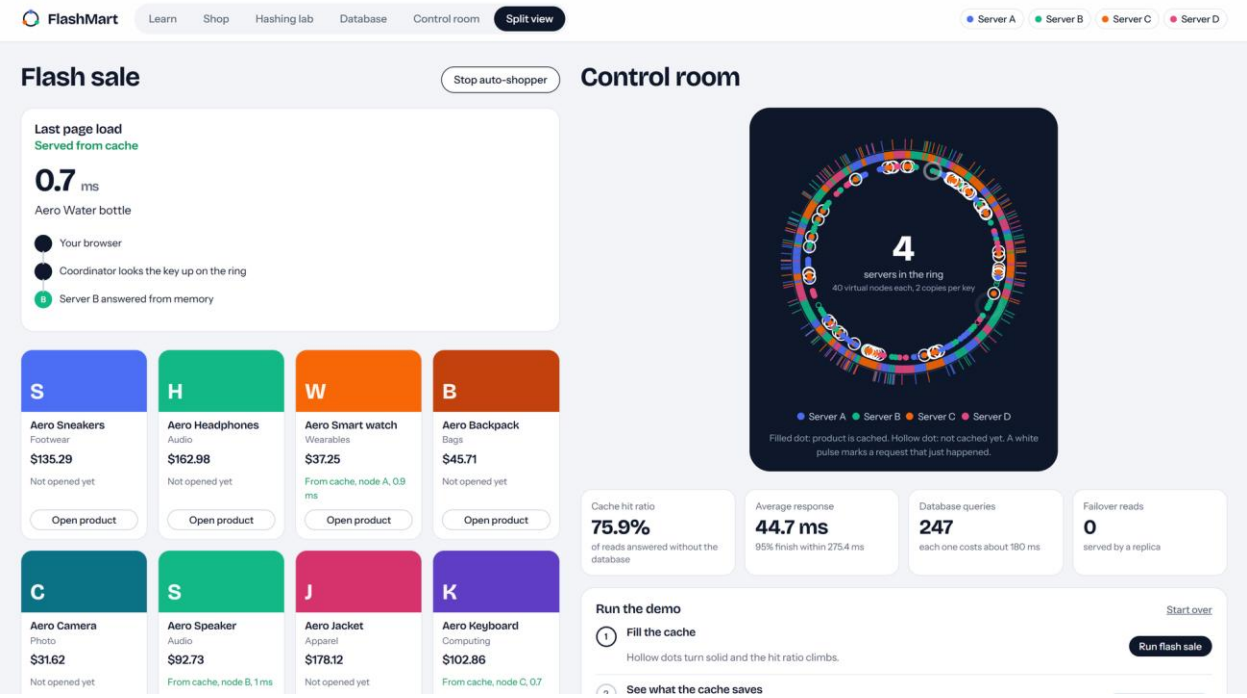


Figure 17: Split view: Shop on the left and Control room on the right, used for presenting. The auto-shopper is running, so the ring shows live request pulses.

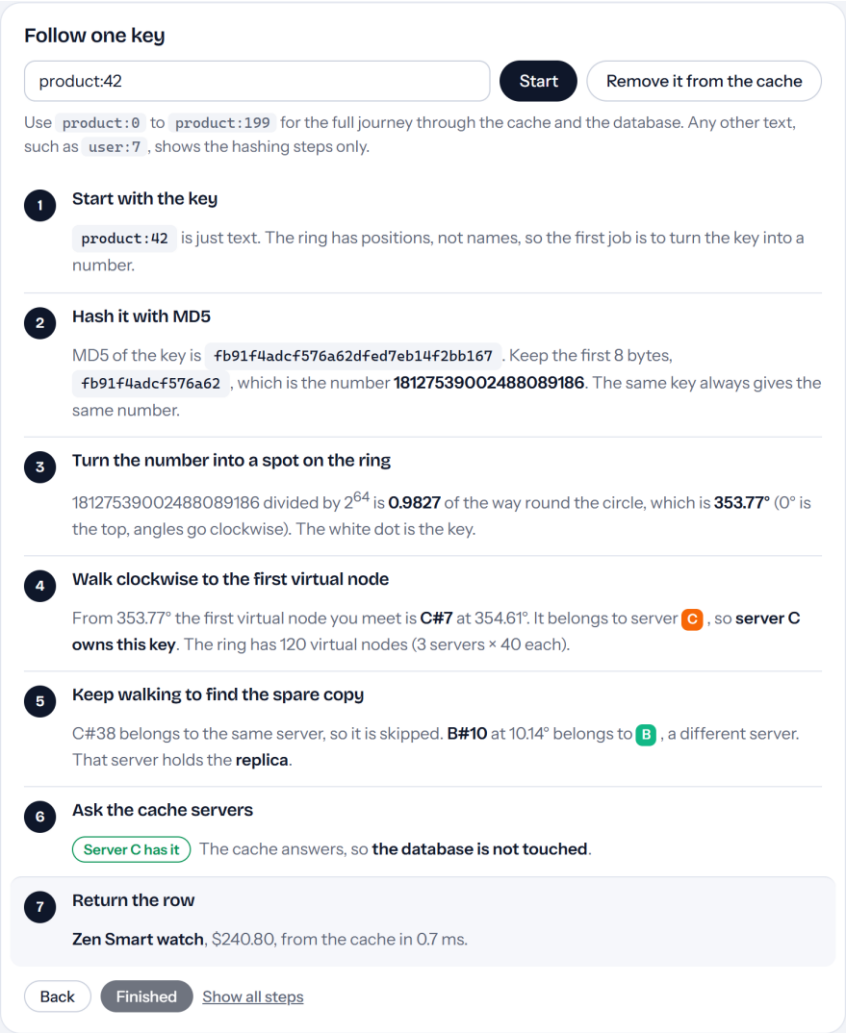
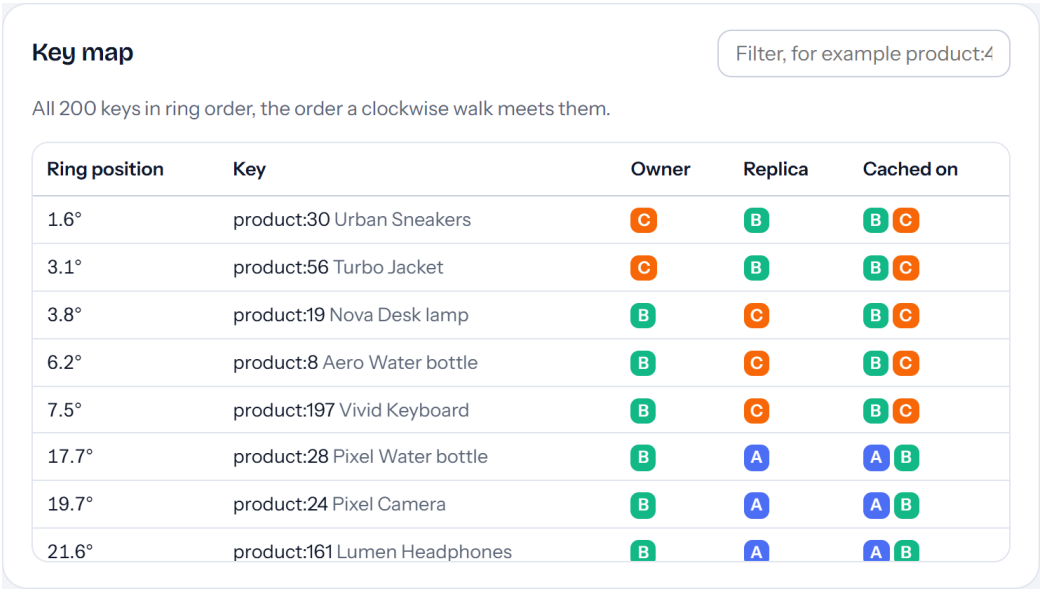


Figure 18: Hashing lab, second lookup of product:42: the cache answers (a cache hit), so the "Query the database" and "Store a copy" steps no longer appear and no SQL is run.



Key map

Filter, for example product:4

All 200 keys in ring order, the order a clockwise walk meets them.

Ring position	Key	Owner	Replica	Cached on
1.6°	product:30 Urban Sneakers	C	B	B C
3.1°	product:56 Turbo Jacket	C	B	B C
3.8°	product:19 Nova Desk lamp	B	C	B C
6.2°	product:8 Aero Water bottle	B	C	B C
7.5°	product:197 Vivid Keyboard	B	C	B C
17.7°	product:28 Pixel Water bottle	B	A	A B
19.7°	product:24 Pixel Camera	B	A	A B
21.6°	product:161 Lumen Headphones	B	A	A B

Figure 19: Hashing lab, key map: all 200 keys in ring order with owner, replica and the servers that currently hold a copy.

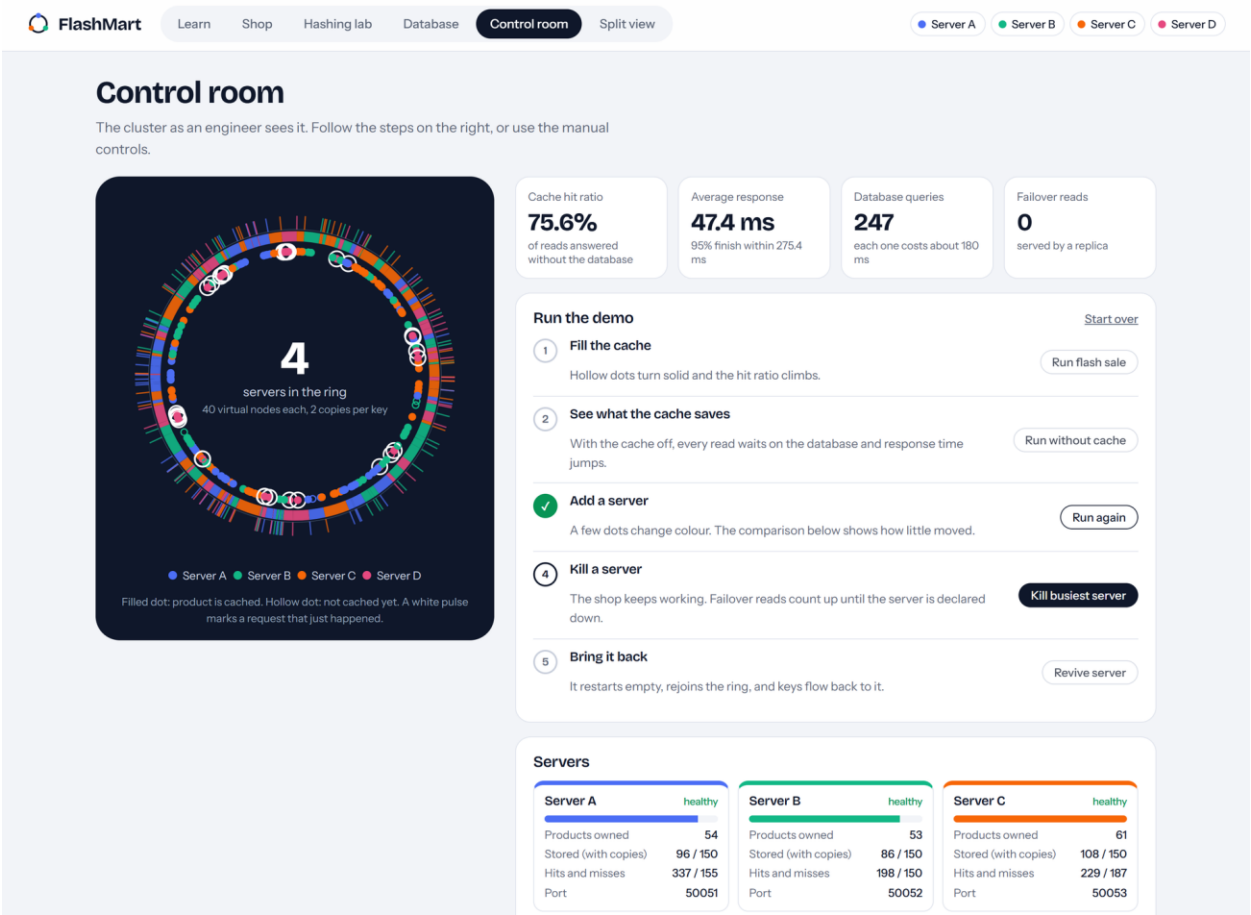


Figure 20: Control room right after the demonstration step "Add a server": the new server D has joined and the keys-moved panel updates.

ts	key	result	server	ms
2026-11-06 10:21:35	product:100	miss	NULL	187.6
2026-11-06 10:21:31	product:100	hit	C	0.8
2026-11-06 10:21:30	product:100	hit	C	0.8
2026-11-06 10:21:23	product:42	hit	C	0.7
2026-11-06 10:21:20	product:42	miss	NULL	188
2026-11-06 10:21:16	product:42	hit	C	0.9
2026-11-06 10:21:16	product:42	hit	C	1.3
2026-11-06 10:21:14	product:52	miss	NULL	262.4

Figure 21: Result of the SQL console preset "Last 10 requests".

Appendix B: Selected pseudocode or code excerpts

Clockwise walk on the hash ring (`hashring.py`): starts at the key's position and stops after the required number of different servers.

```
def walk(self, key, distinct=2):
    start = bisect.bisect_left(self._points, hash_key(key))
    seen, hops = [], []
    for step in range(len(self._points)):
        point = self._points[(start + step) % len(self._points)]
        node = self._owner[point]
        hops.append({"label": self._label[point], "node": node,
                     "angle": angle_of(point), "new": node not in seen})
        if node not in seen:
            seen.append(node)
            if len(seen) == distinct:
                break
    return hops
```

Rebalancing (`cluster.py`, simplified: timeouts and error handling omitted): every key must live on exactly the servers in its preference list.

```
for key in all_keys:
    desired = self.ring.preference_list(key, self.rf)
    holders = [n for n, ks in held.items() if key in ks]
    for node in desired:
        if key not in held.get(node, ()):
            value = self._peek(holders, key)          # internal read, not counted in statistics
            if value is not None:
                self.members[node].stub.Put(pb.PutRequest(key=key, value=value,
ttl_seconds=self.ttl))
    for node in holders:
        if node not in desired:
            self.members[node].stub.Delete(pb.DeleteRequest(key=key))
```

Read path (pseudocode summarising `fetch_product` in `cluster.py`):

```
prefs = ring.preference_list(key, rf)                # owner first, then replica
for name in prefs:
    try:    reply = server[name].Get(key, timeout=0.3)
    except RpcError: continue                          # owner down: try the replica (failover)
    if reply.found: return reply.value                 # cache hit
row = db.query(product_id)                             # cache miss: SELECT on SQLite (about 185
ms)
for name in prefs: server[name].Put(key, row, ttl=300)  # store on owner and replica
db.log_access(key, "hit" or "miss", server, ms)        # one row in access_log
```

Appendix C: Additional test cases / survey instrument

Automated test (tests/test_hashring.py)	What it checks
test_owner_is_deterministic	Two rings built in different orders give the same owner for 2,000 keys.
test_preference_list_distinct_and_starts_with_owner	Owner and replicas are distinct and the owner comes first.
test_replica_count_capped_by_node_count	With two servers only two copies are returned; an empty ring returns none.
test_adding_node_moves_only_a_fraction_of_keys	Adding a server moves 15–35% of keys, and only to the new server.
test_modulo_hashing_moves_most_keys	hash % N moves more than 60% of keys when a server is added.
test_removing_node_only_moves_its_keys	Keys of other servers do not move when one server is removed.
test_more_vnodes_balances_load	200 virtual nodes balance load better than 1.
test_walk_stops_at_distinct_servers_and_labels_vnodes	The walk ends at distinct servers and labels hops like C#7.
test_hash_details_matches_ring_position	The MD5 details shown in the lab equal the position used by the ring.
test_copy_is_independent	A copy of the ring used for previews does not change the real ring.

Appendix D: Repository / demo / deployment information (no credentials or secrets)

Project folder: `DC_microproject` (Python files, `static/`, `templates/`, `tests/`). Run: `python app.py` from the project folder after installing `requirements.txt`; open `http://localhost:8000/`. Inspect the database with `python show_db.py` (add `--tail` for a live feed). Run the tests with `python -m pytest -q tests`. Source repository link: [add repository or drive link if any].

Appendix E: Pseudocode and metric definitions

Failure detection. Every 0.7 s, for each server: send Ping with a 0.4 s timeout. On success reset the missed count and mark the server up. On failure add one to the missed count; after the first miss mark the server suspect; after the second remove it from the ring and rebalance. **Keys moved.** Before and after a membership change, compute the owner of each of the 200 products under the ring and under `hash % N`; the count is the number of products whose owner differs. **Hit ratio** = hits / (hits + misses). **Response time** = time from receiving a request at the coordinator to returning the row, including any database query.

Appendix F: The gRPC contract (cache.proto, abridged)

```
service CacheNode {
  rpc Get(GetRequest) returns (GetReply);          // key, internal flag  ->  found, value
  rpc Put(PutRequest) returns (PutReply);          // key, value, ttl_seconds
  rpc Delete(DeleteRequest) returns (DeleteReply); // key  ->  existed
  rpc Keys(Empty) returns (KeysReply);             // list of stored keys
  rpc Ping(Empty) returns (PingReply);             // name, uptime_ms  (heartbeat)
  rpc Stats(Empty) returns (StatsReply);           // size, capacity, hits, misses, evictions
  rpc Clear(Empty) returns (Empty);
}
```

Appendix G: Items to complete before final submission

Highlighted yellow fields in this document are placeholders. Before submitting:

- Fill in student names, roll numbers, class/division, institute email IDs, group number, faculty mentor and submission date on the first page.
- Confirm the course code, semester and academic year on the first page (copied from the department template used for the reference report).
- Complete the Team Contribution table (Section 5.5) so that it matches the work each member actually did, or delete extra rows for a smaller team.
- Add the date on which each web reference was accessed, and check the citation details in the References list.
- Add a repository or drive link in Appendix D if one exists, and confirm the SDG selection with the mentor.
- If the cache servers are later tested on a second computer, update test TC-11 with the observed result.